O'REILLY LIVE TRAINING · 2 HOURS

Claude Certified Architect – Professional Exam Prep

Cut to the chase for CCAR-P: the blueprint, the decision patterns, and the traps that cost points.

Today's plan

SEGMENT	CLOCK	DOMAINS	WEIGHT
1 · The exam + solution design	0:00–0:25	Blueprint + D1	17%
2 · Models, context, integration	0:25–1:00	D2 + D3	32%
3 · Evaluation + governance	1:00–1:30	D4 + D5	30%
4 · Stakeholders + team enablement	1:30–1:50	D6 + D7	21%
5 · Exam strategy + study plan	1:50–2:00	All	·

Repo: lessons, exercises, 60 practice questions, 60 Anki cards, cheatsheet.

SEGMENT 1 · 0:00 → 0:25 · 25 MIN

The exam and solution design

Exam blueprint · all 7 domains

D1 Solution Design & Architecture · **17%**

Lessons 00 + 01

Exercises 00 + 01

The exam at a glance

ITEM	DETAIL
Format	**63 items** · multiple-choice + multiple-response ("select N")
Time	120 min · **~1 min 54 s per item**
Pass	Scaled **720** on 100–1,000 · 720 ≠ 72% raw
Delivery	**$175** · Pearson VUE, online or test center
Prereqs	**None.** Architect – Foundations is not required
Validity	12 months · free on-time renewal

WHO THE EXAM IS WRITTEN FOR

The minimally qualified candidate

> *"an experienced practitioner who combines an engineering mindset with practical, real-world experience deploying AI solutions."*

- Designs end-to-end: prompts, context, **RAG**, APIs, orchestration, **evals**
- Weighs every choice on five axes: **cost · latency · performance · safety · maintainability**
- Recommended: 3+ years architecture, 6+ months Claude in production

The blueprint: seven weighted domains

#	DOMAIN	WEIGHT	≈ ITEMS
D1	Solution Design & Architecture	17%	11
D2	Claude Models, Prompting & Context Engineering	13%	8
D3	**Integration**	**19%**	**12**
D4	Evaluation, Testing & Optimization	16%	10
D5	Governance, Safety & Risk Management	14%	9
D6	Stakeholder Communication & Lifecycle	14%	9
D7	Developer Productivity & Operational Enablement	7%	4

D1 + D3 + D4 = 52%. Design, integrate, measure.

Six distractor patterns

PATTERN	SOUNDS LIKE
Bigger model	"Switch to a larger model that follows instructions better."
Watch, don't remove	"Add logging." "Add a confirmation step."
Prompt as policy	"Add to the system prompt that it must never..."
Lossy shortcut	"Truncate the document." "Smallest model regardless of fit."
Wrong trigger	"The weights changed." "Temperature is too low."
Over-engineering	"Split it into five specialized agents."

Right answer: the structural fix that matches the trigger and keeps the needed capability.

A team exposes a customer-support agent that can read tickets, draft replies, issue refunds, and delete user accounts. Support staff only ever need to read tickets and draft replies. Applying least-privilege principles, which change best reduces risk?

A. Add logging to the refund and delete tools so misuse can be audited later.

B. Remove the refund and delete tools from the agent's configuration entirely.

C. Keep all tools but add a confirmation prompt before refunds and deletions.

D. Replace the agent with a larger model that follows instructions more reliably.

B · Remove the refund and delete tools entirely

OPTION	PATTERN	WHY IT FAILS
A	Watch, don't remove	Detective control: records damage after it happens
C	Watch, don't remove	Compensating control: capability still reachable by injection
D	Bigger model	Model size doesn't change authorization scope

Remove capability before you guard it.

What the exam asks

- Translate business problems into Claude-based AI solutions
- Design end-to-end architectures (input → processing → output → feedback loops)
- Select appropriate architectural patterns (workflow, agentic, augmented LLM)
- Design multi-agent systems and orchestration strategies
- Apply decomposition techniques for complex problem solving
- Align solutions to business value pillars (efficiency, transformation, productivity, cost, performance SLAs)

Verbatim from the CCAR-P Exam Guide v1.0, Section 6

Four decisions, in this order

1. **Ownership**
   What Claude does, what existing systems do, what humans do
2. **Shape**
   Augment a live interaction · automate a workflow · deploy an agent
3. **Reference architecture**
   Name the pattern you're committing to
4. **Entry point**
   Model, context strategy, surface, delivery route

"Getting these out of order is the most reliable path to an expensive pivot."

Climb the ladder only on evidence

RUNG	WHAT IT IS	WHAT IT COSTS
Augmented LLM	One call + retrieval, tools, memory	Least flexible
Workflow	Predefined code paths	Can't adapt to surprises
Agent	LLM directs its own steps in a loop	Latency, tokens, compounding errors
Multi-agent	Orchestrator + subagents	**~15× the tokens** of chat

If you can draw the flowchart in advance, it's a workflow.

Five workflows, five cues

PATTERN	USE WHEN	CUE IN THE STEM
Prompt chaining	Fixed sequential steps with gates	"extract, then validate, then..."
Routing	Distinct input categories	"simple vs. complex", "by type"
Parallelization	Independent parts, or voting for confidence	"at the same time", "several reviewers"
Orchestrator-workers	Subtasks unknown until input is seen	"varies per request", "across sources"
Evaluator-optimizer	Clear rubric + iteration helps	"style guide", "refine until"

Multi-agent: when it pays

MULTI-AGENT FITS

- Breadth-first research
- Many independent sources
- Information exceeds one context
- Task value pays for ~15× tokens

MULTI-AGENT FAILS

- Tightly coupled work (most coding)
- All agents need the same context
- Real-time coordination
- Assuming subagents share memory

Subagents start with only what you pass them. Give each one its task, constraints, output format, and its own scoped tools.

EXERCISE 1 · OWNERSHIP, SHAPE, PATTERN · ~6 MIN

Five problems, four lines each

- **File:** `exercises/01-pattern-selection.md`
- **Do:** for claims intake, a contact-center copilot, weekly competitor research, a repo migration, and localization, write: ownership · shape · pattern · cost
- **Watch for:** RAG doing a job live state should own (order status), and agents where the steps are known

SEGMENT 1 COMPLETE

Q&A

SEGMENT 2 · 0:25 → 1:00 · 35 MIN

Models, context, and integration

D2 Models, Prompting & Context · 13%

D3 Integration · 19%

Lessons 02 + 03

Exercises 02 + 03 · prompt_caching.py

What the exam asks

- Select appropriate Claude models based on trade-offs
- Design system prompts, templates, and guardrails
- Apply prompt engineering techniques (zero-shot, few-shot, chain-of-thought)
- Optimize context windows and manage token usage
- Implement prompt reuse strategies (caching, modular prompts, Skills)

Verbatim from the CCAR-P Exam Guide v1.0, Section 6

Model choice is a measured trade-off

MODEL (OCT 2026)	FIT	$/MTOK IN · OUT
Fable 5.1	Hardest reasoning, long-horizon agents	10 · 50
Opus 5.5	Agentic coding, knowledge work · "start here"	4 · 20
Sonnet 5.5	Speed + intelligence	2 · 10
Haiku 5.5	Classification, extraction, routing at volume	0.10 · 0.50

Decide on your eval set. Tune effort before switching models. Compare cost per completed task.

An application sends the same 8,000-token system prompt and policy document on every request, followed by a short, varying user message. Latency and cost are both concerns. Which optimization most directly addresses both?

A. Truncate the policy document to the first 1,000 tokens.

B. Switch to the smallest available model regardless of task fit.

C. Place the static system prompt and policy before the dynamic content and enable prompt caching.

D. Move the policy document into a few-shot example block.

C · Static content first, then enable prompt caching

OPTION	PATTERN	WHY IT FAILS
A	Lossy shortcut	Discards required policy
B	Lossy shortcut	"Regardless of task fit"
D	Wrong mechanism	Few-shot teaches format; no cacheable prefix

Caching matches an exact prefix: tools → system → messages, static → dynamic.

Prompting and context: the decision table

NEED	USE
Exact format or tricky mapping	3–5 diverse **few-shot** examples + **structured outputs**
Multi-step reasoning	Higher **effort** / reasoning before the answer; arithmetic to a tool
Guaranteed JSON shape	**Structured outputs** or strict tool use (prefill and forced `tool_choice` error on new models)
Long document Q&A	Document first, question last, quotes before answer
Long agent sessions	Context editing · compaction · memory tool · a persistent facts block
Knowledge base < ~200k tokens	All in context, cached. Skip RAG
Many reusable procedures	**Agent Skills** (progressive disclosure, ~100 tokens each until used)

🔴 EXERCISE 2 · BREAK THE CACHE · ~4 MIN

One timestamp, three layouts

- **Run:** `uv run exercises/scripts/prompt_caching.py`
- **Watch:** `cache_write` vs. `cache_read` per call
- **Layouts:** static prefix → timestamp first → timestamp after the cached block

```
1) cache_write=2483  cache_read=     0
   cache_write=   0  cache_read=  2483
2) cache_write=2511  cache_read=     0   (every call)
3) cache_write=   0  cache_read=  2483
```

What the exam asks

- Evaluate tool/agent configuration for capability bloat
- Analyze authentication and authorization requirements to identify security gaps
- Evaluate accuracy-latency trade-offs and justify configuration decisions
- Analyze observability challenges and select monitoring strategies at scale
- Design a RAG pipeline with appropriate chunking and indexing strategies
- Apply retrieval strategies matched to data shape and query pattern
- Evaluate connection protocols (MCP, API/CLI, agent-to-agent)
- Evaluate progressive discovery vs. monolithic context strategy

Verbatim from the CCAR-P Exam Guide v1.0, Section 6 (last two abridged)

Tools and identity: find the gap

FINDING	FIX
50+ tools, wrong picks, context burned	Scoped sub-agents · consolidate · **tool search** with `defer_loading`
Tools the role never uses	**Remove them**
Shared admin service account	Per-user delegated OAuth tokens bound to the session
Identity from model text (`user_email` arg)	Identity from SSO; authorize server-side
MCP token passthrough	Validate tokens issued for the server; own downstream creds
API key in the browser or prompt	Backend proxy; secrets stay server-side
JIT provisioning only	SCIM, so IdP removal deprovisions

RAG: design the pipeline

STAGE	EXAM ANSWER
Before RAG	< ~200k tokens (~500 pages)? Long context + caching
Chunking	Structure-aware: headings, whole tables and procedures
Contextual retrieval	50–100 tokens of context per chunk: failures **−35% / −49% / −67%**
Index	**Hybrid**: embeddings + BM25 + metadata filters
Retrieve	Wide (top-20), then **rerank**
Refresh	Re-index with the same embedding model; re-run retrieval evals

Match retrieval to the data

DATA / QUERY	STRATEGY
Docs, conceptual questions	Hybrid embeddings + rerank
Part numbers, error codes, IDs	**BM25** + exact-match filters
"Q3 sales by region"	**Query tool** (read-only SQL / API)
Order status, balance, inventory	**Tool call** to the system of record
Permission- or date-scoped	Metadata filters at retrieval
Multi-hop	Agentic, iterative search

Retrieval over a nightly index is the wrong owner for live state.

API, MCP, or agent-to-agent?

API / CLI

One consumer. Deterministic call at a known step in code.

Not when many clients need it

MCP

One integration reused by many Claude clients. Tools, resources, prompts. stdio local · Streamable HTTP + OAuth remote.

Not for one fixed call

AGENT-TO-AGENT

Independent agents, often cross-org, each owning tools, state, and policy.

Not when a tool would do

Progressive discovery (tool search, Skills, retrieval) beats **monolithic context** for large catalogs with sparse use. Monolithic wins for small, static, always-needed context. Cache it.

Observability: what dashboards miss

SIGNAL	TRAP
`stop_reason: "refusal"`	Arrives as **HTTP 200**. Instrument separately; configure fallback per request
429 rate limit	Honor `retry-after`; ramp traffic gradually
429 from a spend cap	No `retry-after`. Retries keep failing. Alert
529 overloaded	Backoff, fallback model or queue
Cache hit rate drops	Silent cost spike

Trace every step with one request ID: prompt version, retrieved chunks, tool calls + args, tokens, latency. Redact PII.

🔴 EXERCISE 3 · INTEGRATION DESIGN REVIEW · ~6 MIN

Seven flaws in one IT/HR assistant

- **File:** `exercises/03-integration-review.md`
- **Design:** 47 tools on 6 MCP servers · one shared admin account · rules in the prompt · 1,800 pages in the system prompt · vacation balance from policy docs · a partner agent wrapped as a tool · final-answer logs only
- **Do:** name the problem, the objective it violates, and the fix

Technicians search with part numbers such as "XR-7741-B" and error codes. Embedding-only retrieval returns similar but wrong parts. Which change fits the query pattern?

A. Add keyword (BM25) retrieval in a hybrid setup with exact-match filters on identifiers, then rerank.

B. Switch to a larger embedding model so identifiers are encoded more precisely.

C. Increase chunk size so each chunk covers more part numbers.

D. Instruct the model to double-check that part numbers match before answering.

A · Hybrid retrieval with BM25 and exact-match filters

OPTION	PATTERN	WHY IT FAILS
B	Bigger model	Similarity still blurs near-identical codes
C	Lossy shortcut	Larger chunks dilute relevance
D	Prompt as policy	Can't verify what retrieval never returned

SEGMENT 2 COMPLETE

Q&A + 5-minute break

SEGMENT 3 · 1:00 → 1:30 · 30 MIN

Evaluation and governance

D4 Evaluation, Testing & Optimization · **16%**

D5 Governance, Safety & Risk · **14%**

Lessons 04 + 05

Exercises 04 + 05 · eval_harness.py

What the exam asks

- Define evaluation metrics (accuracy, latency, cost, safety, security)
- Design evaluation datasets and test frameworks using mixed methodologies
- Conduct A/B testing and iterative improvements
- Diagnose system issues (prompt failure, hallucinations, model mismatch)
- Optimize token usage, latency, and cost-performance trade-offs
- Monitor system performance using logging and observability tools

Verbatim from the CCAR-P Exam Guide v1.0, Section 6

Evals are the gate

GRADER	USE FOR	RULE
Code-based	Labels, JSON schema, exact match, tests	Fastest, most reliable. Prefer it
LLM judge	Faithfulness, tone, rubric scoring	Detailed rubric · reason then score · **validate vs. humans**
Human	Calibrating the judge, high-stakes spot checks	Gold standard. Avoid at scale

- **Criteria are measurable:** "≥ 90% score ≥ 4/5, p95 < 5 s, < $0.01", not "good summaries"
- **Volume over hand-graded perfection.** Hold out a set you never tune on
- **pass^k** for customer-facing agents: 75% per run over 3 runs ≈ 42%

A RAG system suddenly returns confident but incorrect answers after a document refresh, while latency and model version are unchanged. What is the most likely first place to investigate?

A. The model weights have silently changed.

B. The retrieval/indexing step is returning irrelevant or stale chunks.

C. The temperature setting is too low.

D. The context window has shrunk.

B · Retrieval/indexing returns irrelevant or stale chunks

OPTION	PATTERN	WHY IT FAILS
A	Wrong trigger	The stem rules out a model change
C	Wrong trigger	Temperature doesn't cause wrong facts after a refresh
D	Wrong trigger	Windows don't shrink

Diagnose from what changed.

Optimize cost per completed task

LEVER	SAVES	WATCH
Prompt caching	Input cost + time to first token	Static first
Message Batches API	**50%**	Async, up to 24 h. Not for users waiting
Model routing	Cost + latency	Verify the hard tail on evals
Effort	Cost + latency	Raise only where evals show gains
Output length	Output tokens cost 5× input	Tight `max_tokens`, structured output
Streaming	Perceived latency only	Total time and cost unchanged

EXERCISE 4 · EVALS AS THE GATE · ~5 MIN

Should we switch to the smaller model?

- **File:** `exercises/04-eval-plan.md` (contract extraction plan)
- **Run:** `uv run exercises/scripts/eval_harness.py`
- **Watch:** code-graded accuracy, LLM-judge score, p50 latency, cost per model
- **Decide:** does the candidate pass your gate?

What the exam asks

- Implement guardrails and safety controls
- Identify risks, limitations, and failure modes of LLM systems
- Apply human-in-the-loop validation strategies
- Ensure compliance with regulations (e.g., GDPR, HIPAA, FedRAMP)
- Address ethical AI considerations (bias, fairness, transparency)

Verbatim from the CCAR-P Exam Guide v1.0, Section 6

The safety stack fails closed

LAYER	CONTROL	TYPE
Input	Injection/jailbreak screen, PII redaction	Model-based + rules
System prompt	Scope, "say I don't know"	**Guidance only**
Tool calls	Least privilege, per-user authz, parameter limits, approvals	**Deterministic**
Output	Schema, PII leak, grounding, policy	Both
Audit	Inputs, tool calls, outputs, reviewer decisions	Deterministic

Training reduces harm. Your application enforces your rules. When a control errors, block or route to a human.

Review routing and compliance

ROUTE REVIEW BY

- **Confidence**, calibrated on labeled data (not self-reported)
- **Reversibility** of the action
- **Cost** of a wrong answer
- Automate + sample low risk; approve the irreversible

MAP EVERY OBLIGATION

- Obligation → **control** → **owner** → **evidence**
- **GDPR:** minimize, residency/transfer + DPA, human review of legal-effect decisions
- **HIPAA:** BAA before PHI; check feature eligibility
- **FedRAMP High:** Bedrock GovCloud or Vertex AI

Fairness is measured by outcomes per group. Deleting a column proves nothing.

A procurement agent approves purchase orders. Policy says orders over $10,000 need a director's approval. The rule is in the system prompt, yet logs show three violations last month. What is the best fix?

A. Restate the rule in capital letters at the top and bottom of the system prompt.

B. Enforce the threshold in the approval tool or a pre-tool check: calls over $10,000 are blocked and routed to a director's queue.

C. Log every approval over $10,000 for the monthly audit.

D. Switch to a more capable model that follows instructions more reliably.

B · Enforce the threshold in code, outside the model

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	Louder is still best effort
C	Watch, don't remove	Records violations after they happen
D	Bigger model	Still best effort

A rule that must always hold lives in code, permissions, or hooks.

SEGMENT 3 COMPLETE

Q&A

SEGMENT 4 · 1:30 → 1:50 · 20 MIN

Stakeholders and team enablement

D6 Stakeholder Communication & Lifecycle · **14%**

D7 Developer Productivity & Enablement · **7%**

Lessons 06 + 07

Exercises 06 + 07

Three meetings decide whether it ships

DISCOVERY

Process, baseline, volume, latency tolerance, cost of errors, data, constraints, the metric. **Write assumptions down.**

APPROVAL

Each option: **cost** · **risk** · **reversal**. A recommendation. An ADR.

HANDOFF

Diagrams, ADRs, prompt registry, **eval suite**, **runbooks**, owners.

Never open discovery with technology. Never promise "always correct."

Lifecycle: plan for change

EVENT	ARCHITECT'S MOVE
Model deprecation notice (≥ 60 days)	Eval the successor · re-baseline cost and latency · migrate before retirement
Eval score drop	Review trigger: diagnose, iterate, re-run evals
SLA breach	Fallback, page owner, fix by deadline
Procurement says "must run in our AWS"	Bedrock first; constraints before latency or cost
Pilot stalls	Agree graduation criteria up front: performance, operations, risk

Which layer does a setting belong at?

SCOPE	PUT HERE
Managed (no opt-out)	Security policy: deny rules, MCP allowlist, no bypass mode
Project (committed)	`CLAUDE.md` conventions, `.claude/settings.json`, hooks, `.mcp.json`, skills
Local (gitignored)	One developer's overrides
User	Personal preferences

CLAUDE.md is guidance. Permissions, hooks, and managed settings are enforcement.

Security requires that no developer can run Claude Code in bypass-permissions mode or connect unapproved MCP servers. Where should this be configured?

A. In each repository's CLAUDE.md file.

B. In managed (enterprise) settings deployed to every machine.

C. In each developer's ~/.claude/settings.json.

D. In the team wiki's onboarding checklist.

B · Managed settings: highest precedence, no opt-out

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	Guidance, not enforcement
C	Wrong scope	Developers can edit their own files
D	Prompt as policy	Documentation is not enforcement

SEGMENT 4 COMPLETE

Q&A

SEGMENT 5 · 1:50 → 2:00 · 10 MIN

Exam strategy and study plan

All domains

Read the trigger, not the options

CUE IN THE STEM	POINTS TO
"only ever need"	Remove tools / permissions
"must always", "never"	Code, hooks, permissions, schema
"same prompt every request"	Caching, static first
"overnight", "not user-facing"	Batch API
"after a document refresh"	Retrieval / indexing
"after switching models"	Re-run evals; mismatch or prompt drift
"steps are known"	Workflow, not agent
"live status", "balance"	Tool call, not RAG
"across organizations"	Agent-to-agent
"every developer, no opt-out"	Managed settings

Three weeks, weighted by the blueprint

WEEK	DOMAINS	DO
1	D1 · D2 · D3 (49%)	Lessons 01–03 · exercises 00–03 · caching script · flashcards daily
2	D4 · D5 (30%)	Lessons 04–05 · exercises 04–05 · eval harness · Q1–Q30 timed
3	D6 · D7 + review (21%)	Lessons 06–08 · exercises 06–07 · Q31–Q60 timed · all 60 again

Build one small end-to-end system with RAG, evals, and logging. The guide recommends it.

If you remember seven things

1. **Simplest pattern that works.**
   Workflow before agent, agent before multi-agent, only on evidence.
2. **Remove capability before you guard it.**
   Least privilege beats logging and confirmations.
3. **Enforce in code, not in the prompt.**
   Permissions, hooks, schemas, validators.
4. **Static first, then cache.**
   Exact prefix. Batch what can wait.
5. **Diagnose from what changed.**
   Refresh → retrieval. Model swap → evals.
6. **Evals gate every change.**
   Measured criteria, validated judges, cost per task.
7. **Design for the handoff.**
   Cost, risk, reversal. Runbooks and owners.

RESOURCES · Q&A

Thank you. Questions?

- Official Exam Guide v1.0 (PDF)
- Official prep path · 5 courses
- Building effective agents · Contextual Retrieval
- Course repo · lessons, exercises, 60 questions, 60 Anki cards

Lucas Soares
O'Reilly Live Training